2026年西安交通大学
形式语言与编译

前 言

- 计算机是“二进制”机器，人提供给它的是高级语言程序，这就造成人与计算机在交流上存在 “鸿沟”。
- 为此编译器发挥着不可替代的作用。
- 在需求牵引和技术推动之下，多样化的计算机硬件成果和为数众多的高级语言加剧了这一“鸿沟”。
- 这就让编译器变得越来越复杂。

经典视觉

- 典型编译器如GCC 与 LLVM 都是超大规模的编译器基础设施项目，代码行数均以千万级计，且持续增长。
- 在此背景下，编译课程教学大多注重理论落地，强调工程能力的培养。

AI编程曙光初现

- 随着可以自动生成代码的AI编程助手的出现，程序开发的生产力得以空前提升。在这样一种程序“爆炸”场景下，更深层次的编译反馈以及更为优化的代码生成方法成为新的编译挑战。在高性能计算、可信计算、网络空间安全等领域，从编译器中获取功能并作为责任移交到程序员的手里，展现出编译知识更为广泛的应用场景。

AI编程快速发展

- AI编程挑战程序员已成为事实。回顾AlphaGo碾压人类棋手事件。
- 辛顿：AI正在取代部分编程任务，重塑编程行业。
- Anthropic公司“氛围式编程”完成完整C编译器并通过测试。LLVM发明人据此认为借助AI基于现有设计文档做代码开发和测试越来越容易并可自动化。
- 斯坦福CS146S“The Modern Software Developer”教授“借助AI Agent完成软件工程全流程”

本课程强调理论性

- 在事实上，今天的编译器已经成为一种庞大的不可替代的复杂软件，是工业基础软件高地之一。
- 在AI能够写出“你想要的”代码的时候，对人而言，“你想要的”这种抽象能力变得更为重要，相应地直接写编译器代码这种传统意义的工程能力其重要性可能有所下降。
- 因此，面对原理、方法、技术、实现等多方面编译知识，将强调理论性作为本课程的一个重要属性，有助于应对复杂细节，提升针对共性问题领悟到的思维创新能力。

本课程特点

- 目的是为了缓解编译课程普遍存在的“难学”制约能力的问题。
- 其一，采用文法作为一条确定的、可扩展的主线来组织知识，避免出现常见的编译知识“碎片化”现象造成思维不连续困惑。
- 其二，提供一套符号系统，作为形式化描述知识的手段，有助于思维的逻辑性、抽象性和一致性，并以此提升学习效率。
- 其三，强调从“为什么”方面来表达知识，有助于学生在学习过程中进行主动思考。

各章安排

- 第1-3、5、7章介绍Ⅱ、Ⅲ型形式语言理论，描绘抽象意义下的语言识别器、判定性质及等价性质。
- 第4、6、8章介绍词法、语法分析，描绘语言识别器的可组合、可转换原理以及实现。
- 第9、10章包含属性文法、符号表、中间语言和语法制导的语义分析方法，描绘形式与内涵一致的、符号化的语义分析原理。
- 第11章介绍运行时环境和函数的可执行代码，描绘最基本的可执行代码框架及构建方法。第12章简要介绍程序分析和寄存器分配算法*。

教学安排

- 综合成绩评定
  - 平时成绩 30%，包括作业、课堂表现、到课情况
  - 期末考试 70%
- 授课方式
  - 线下：见课表安排
  - 思源学堂：师生对接处，对接课程目标、课件、课程资料等信息，客观题作业发布与提交（待定）
  - 雨课堂：发布作业、公告等，同学提交作业
  - 微信群：常规联系处，通知、公告、反馈、集体答疑等
  - 微信私信：具体问题讨论、答疑等
  - 另外有任何联系需要都可发邮箱zhaoy@xjtu.edu.cn

关于课程的作业

- 完成老师即时布置的作业，预计总共7次左右。
- 注意，对于雨课堂的主观题作业，统一要求采用非图片PDF文件方式提交。
- 建议使用LATEX、WORD等写作业。应善于使用“自动校对”功能来提高输入符号及带格式字符的效率。

教材及参考书

- 赵银亮 形式语言与编译，西安交通大学，2026年，ISBN 978-7-5693-2653-6

- John E. Hopcroft等. Introduction to Automata Theory, Languages, and Computation (3rd Ed.中文版，孙家骕等译)，机械工业出版社，2008。
- Torben Ægidius Mogensen. Introduction to Compiler Design (2nd edition). Springer International Publishing AG, 2017.

先修课程

- 为了最好地修读本课程：
  - 读者应当学习过关于“离散数学”、“数据结构”和“程序设计”课程；
  - 熟悉集合论与代数系统，熟悉常见数据结构如序列、表、树、图等，熟悉C语言并有一定的程序设计经验；
  - 熟悉数学归纳法和逻辑推理等定理证明技巧；
  - 若对于Pascal、Fortran和Lisp语言有初步了解会更有利于学习。
- 建议对于用到的先修课知识点（见课程资料）有所熟悉。

第1章 绪论

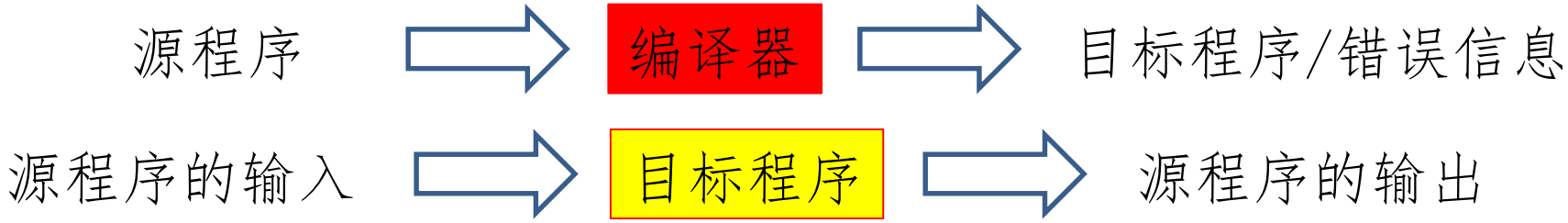
- 编译概述
- 图灵机、冯·诺依曼结构
- 形式语言概论
- 本课程的主文法及符号系统

1.1编译概述

- “编译”是一种可具体化的抽象概念。
- 具体化为某个编译程序，也称语言处理器、编译器、编译系统等。
- 具体化为某种程序转换过程，称为编译过程，如5阶段编译过程、编译遍等。
- 有理论体系和方法论：2型和3型形式语言理论（多语言联合DFA、itemPDA等）、LL(k)和LR(k)分析法、属性文法与语法制导翻译、运行时环境、代码优化、流分析、寄存器分配等

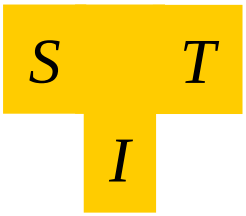
编译程序

- 一个编译程序本身是个程序，其功能是将用某个语言写的程序（称为源程序）作为输入， 并翻译成为功能上等价的另一个语言程序（称为目标程序）。
- 用来写源程序所用的语言，即源语言，通常是高级语言，如C，FORTRAN等。
- 用来写目标程序所用的语言，即目标语言，通常是低级语言，如汇编语言或机器语言。
- 随着翻译过程的进行，编译程序也报告错误信息以帮助程序员改错，直到翻译通过为止。



编译器的T型图表示

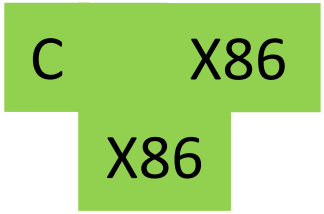
源程序 ⇨ 编译器 ⇨ 目标程序



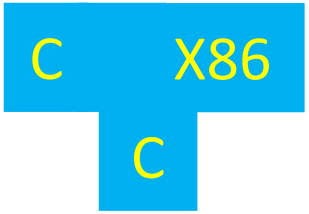
- S – 源语言（高级语言）
- T – 目标语言（低级语言）
- I – 实现语言

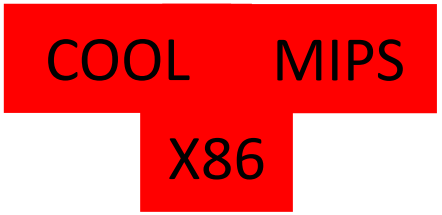
用 I 语言写的编译器被用于将 S 语言写的源程序翻译成用 T 语言写的目标程序

.c



.o





问：编译的多上下文场景如何？

高级语言

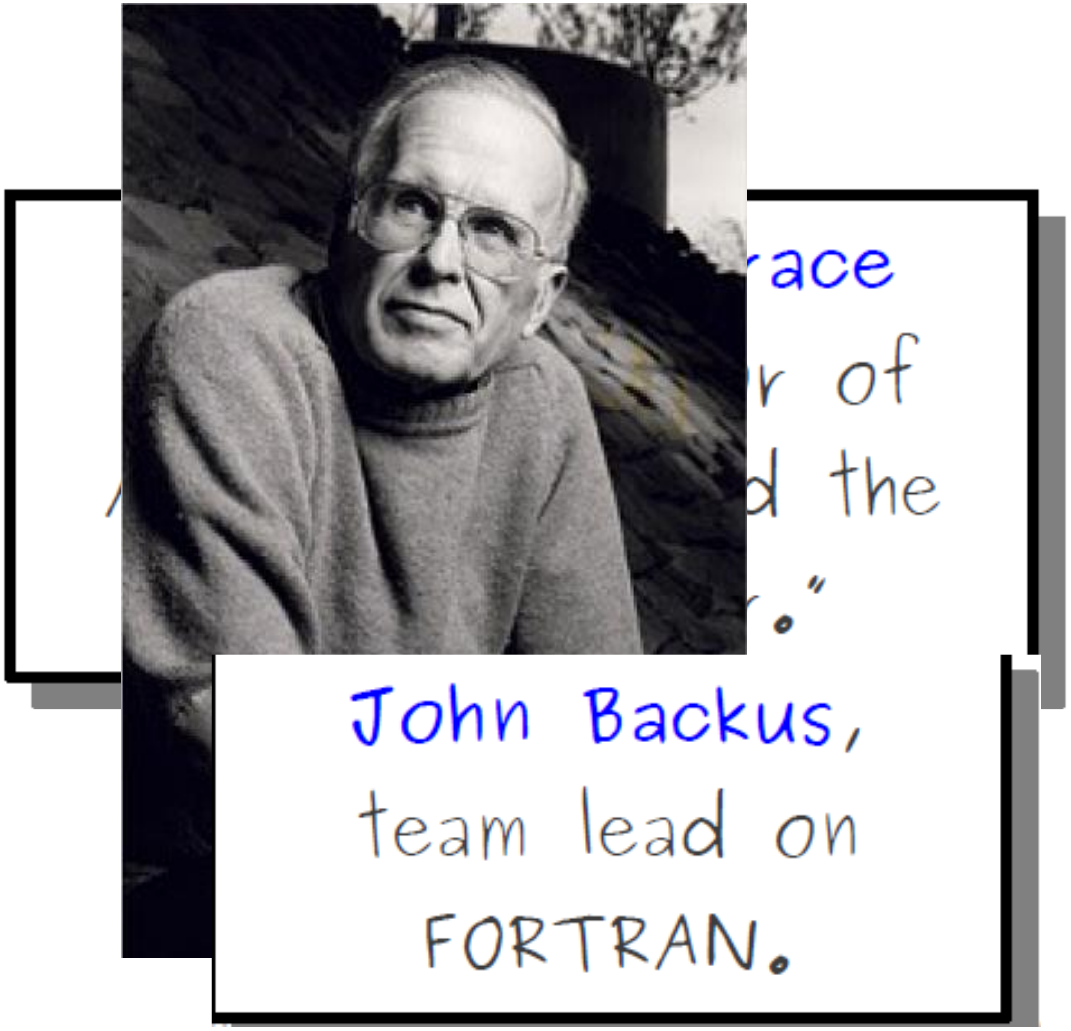


Image: http://upload.wikimedia.org/wikipedia/commons/thumb/5/55/Grace_Hopper.jpg/300px-Grace_Hopper.jpg
http://www.nytimes.com/2007/03/20/business/20backus.html

分层认识编程语言

ELF/COFF

```
#as globe var
.data
msg:   .ascii "Hello World"

#as main
.text
la $a0 msg
li $v0 4
syscall
```

```
1  #include <stdio.h>
2
3  main() {
4    printf("hello world\n");
5  }
```

- 低级语言（硬件直接执行）：指令、地址、RAM；
- 高级语言（命令式与陈述式等）（名字、值）。
- 高级语言数千种每年还有近百种发明。
- 对编译器的影响：要求适应新语言、利用新硬件、不断提升性能

编译器示意

目标语言是高级/低级语言时？
实现语言是高级/低级语言时？
目标机与宿主机相异时？ 交叉编译器

目标程序

```
#as globe var
.data
msg:   .ascii "Hello World"

#as main
.text
la $a0 msg
li $v0 4
syscall
```

错误信息

```
1  #include <stdio.h>
2
3  main() {
4    printf("hello world\n");
5  }
```

编译器

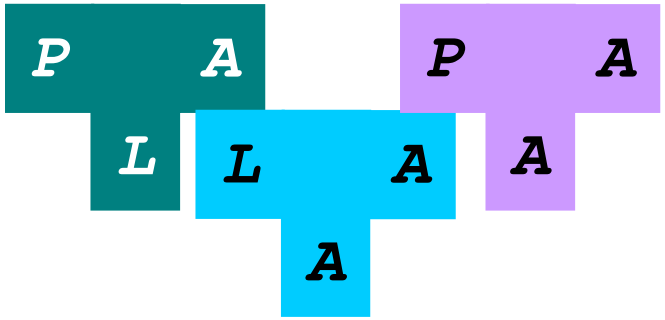
源程序

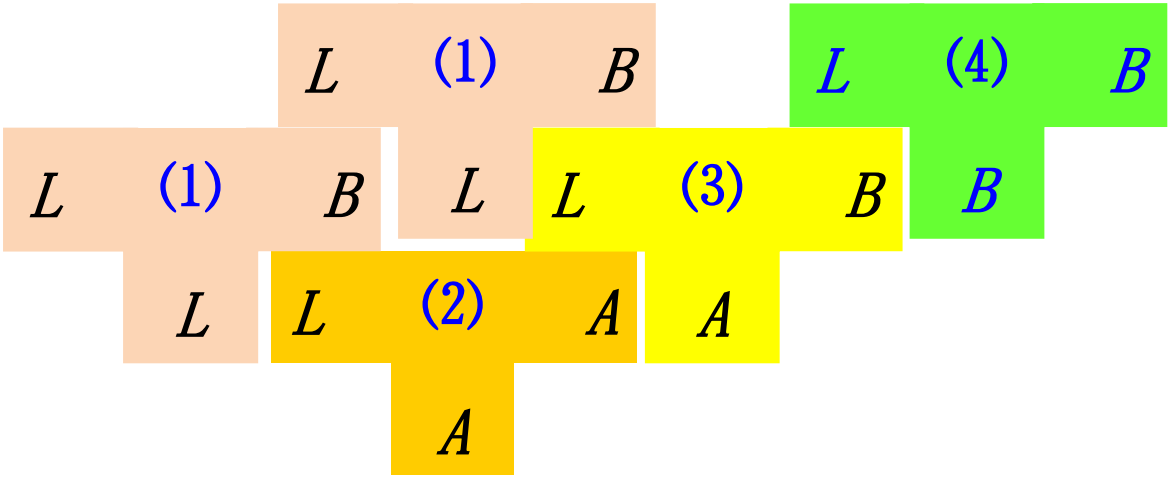
源语言

实现语言

目标语言

用T型图表示移植编译器的过程





给定用*L*写的*P*语言编译器，使用*L*语言的*A*机器编译器进行编译，得到*A*机器的*P*语言编译器

将编译器(1)利用编译器(2)移植为编译器(4)

编译过程与编译程序

源语言　　实现语言　　目标语言

无　　宿主机器　　目标机器

✓ 涉及到三个程序，各程序是用什么语言写的？运行的机器是什么？

编译过程

源程序（字符串）

词法分析
词法记号串（词法单元及其种属）
语法分析
语法树（句子、终结符、非终结符）
语义分析
AST/符号表+中间代码
代码优化
优化的中间表示
代码生成
目标程序（可执行代码）

Step1 词法分析

```
int fact(int n;) {
   if (n<2) return 1 else return n*fact (n-1,)};
print fact (5,)
```

（第2行）⟨IF, _⟩⟨LPA, _⟩⟨ID, n⟩⟨ROP, <⟩⟨NUM, 2⟩⟨RPA, _⟩ ⟨RETURN, _⟩⟨NUM, 1⟩⟨ELSE, _⟩⟨RETURN, _⟩⟨ID, n⟩⟨AOP, *⟩⟨ID, fact⟩⟨LPA, _⟩⟨ID, n⟩⟨AOP, -⟩⟨NUM, 1⟩⟨CMA, _⟩⟨RPA, _⟩⟨RBR, _⟩⟨SCO, _⟩

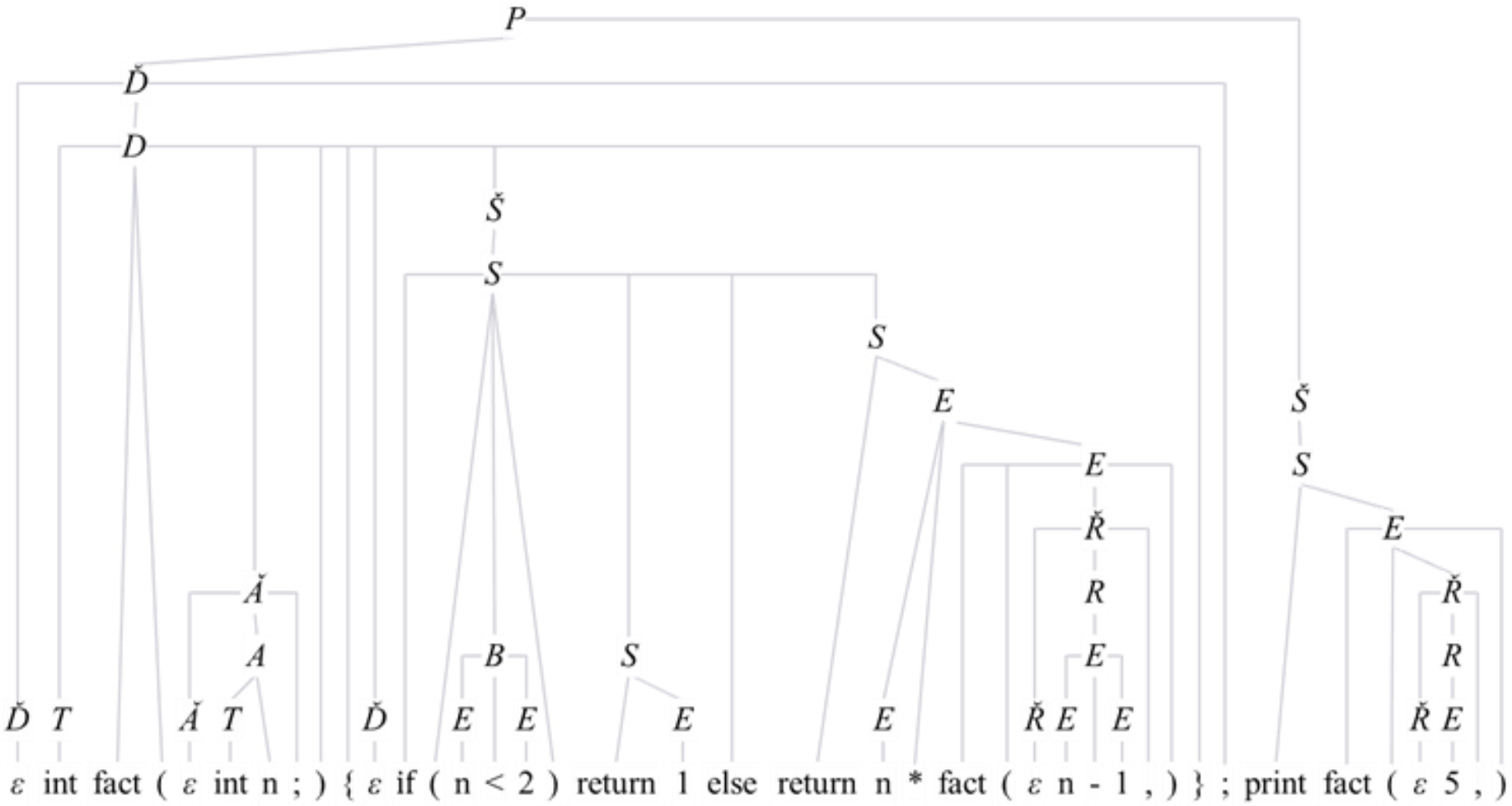
词法记号（Token）是词法种属和词法单元（lexeme）二元组。

词法种属：整数、浮点数、标识符、关键字、运算符等等。

Step2 语法分析

- 将词法分析的输出（记号串）作为语法分析的输入（终结符串），依据语言的语法规则，检查是否符合上下文无关文法，并建立程序的层次结构，将语法树或AST作为输出。
- 发现语法错误若有。
- 本课程默认使用主文法。

Fact程序的语法树


P
Ď
D
Š
S
S
E
E
Ř
R
E
Ř E E
Ď T
Ă T
Ď
E E
E
E
Š
S
E
Ř
R
Ř E
ε int fact (ε int n ;) { ε if (n < 2) return 1 else return n * fact (ε n - 1 ,) } ; print fact (ε 5 ,)

带注释语法树片段

```
26_____if
27_____(
28_____B tc: (l1); fc: (l2); code:[IF n==1 THEN l1 ELSE l2]
29_____E                    place:n; code:[]
30_____d<n>
31_____r<==>
32_____E                    place:1; code:[];
33_____i<1>
34_____)
35_____S                    code:[RETURN a]
36_____return
37_____E                    place:a; code:[]
38_____d<a>
39_____else
```

例：发现语法错误

```
int * foo(i, j, k))
    int i;
    int j;
{
    for(i=0; i j) {
    fi(i>j)
    return j;
}
```

多余的括号

~~类型未声明~~

缺少步长

不是表达式

不是关键字（每个语言的关键字都是确定的）

Step3 语义分析

- 为语法分析识别的各个语法单元，获取其语义。
- （1）从程序声明中提取语义信息并登记到符号表里。
- （2）对程序语句都生成中间代码并以函数为单位组织起来。
- （3）发现语义错误若有。
- 通常把语义分析阶段分为语义分析及中间代码生成两个子阶段，前者着重类型检查和从声明获取语义信息，后者生成中间代码（如三地址指令）。
- 语义分析结果为AST表示或“符号表+三地址代码”形式。

符号表+中间代码

@table: (outer:NIL width:16 argc:0 arglist:NIL rtype:INT level:0

code:[t4 = 1; PAR t4; t5 = CALL fact, 1; PRINT t5]

entry: (name:fact type:FUNC offset:8 mytab:fact@table)
entry: (name:t4 type:TEMP offset:12)
entry: (name:t5 type:TEMP offset:16))

fact@table: (outer:@table width:16 argc:1 arglist: (n) rtype:INT level:1 code:[IF n<2 THEN l1 ELSE l2; LABEL l1; RETURN 1; GOTO l3; LABEL l2; t1 = n-1; PAR t1; t2 = CALL fact,1; t3 = n*t2; RETURN t3; LABEL l3]

entry: (name:n type:INT offset:4) entry: (name:t1 type:TEMP offset:8)… entry: (name:t3 type:TEMP offset:16))

例：发现语义错误

```
int * foo(i, j, k)
    int i;
    int j;
{
    int x;
    x = x + j + N;
    return j;
}
```

类型未声明

变量未声明

返回结果类型不匹配

使用了没有初始化的变量

函数的运行时代码/可执行代码雏形

运行时环境

```
fact@label:
[SP=SP-16; M[SP]=FP; M[SP+8]=LR; //序言，保存返址、主调栈帧。
FP=SP; //调用序列，序言，置被调的栈指针
n = M[SP+16]; //序言，利用栈传递参数到形参
SP=SP-16//序言，分配局部区
IF n<2 THEN l1 ELSE l2; LABEL l1;
M[FP+16]=1; GOTO lepi; //返回序列，栈传结果、转尾声
GOTO l3; LABEL l2; t1 = n-1;
SP=SP-16; M[SP]=t1; //调用序列，利用栈传递参数
BL fact; SP=SP+16; t2=M[SP]; //调用序列，返回后取返回值
t3 = n*t2;
M[FP+16] = t3; GOTO lepi; //返回序列，栈传结果、转尾声
LABEL l3;
LABEL lepi; SP=SP+16; //返回序列，尾声，释放局部区
SP=FP; FP=M[SP];LR=M[SP+8]; //返回序列，尾声，恢复FP、返址
RET] //返回序列，尾声，按返址返回
```

Step4 代码优化

- 代码优化的内容较多，可用于中间代码至输出目标代码之间的代码形式。因此可以在可执行代码雏形上做优化，例如寄存器分配、局部优化等，最后（经Step5）生成完整的ARM64代码。

```
fact:
    stp     x29, x30, [sp, -16]!
    mov     x29, sp
    cmp     w0, 2
    b.ge    .Lrecursive
    mov     w0, 1
    b       .Lreturn
.Lrecursive:
    str     w0, [sp, -16]!
    sub     w0, w0, 1
    bl      fact
    ldr     w1, [sp], 16
    mul     w0, w1, w0
.Lreturn:
    ldp     x29, x30, [sp], 16
    ret
main:
    mov     w0, 5
    stp     x29, x30, [sp, -16]!
    mov     x29, sp
    bl      fact
    …     //打印w0的代码
    ldp     x29, x30, [sp], 16
    ret
```

其他编译器话题

- 错误处理程序
- 符号表管理
- 编译遍
- 一遍与多遍编译
- 前端、后端、中间端
- 构造工具：LEX、YACC等
- 应用

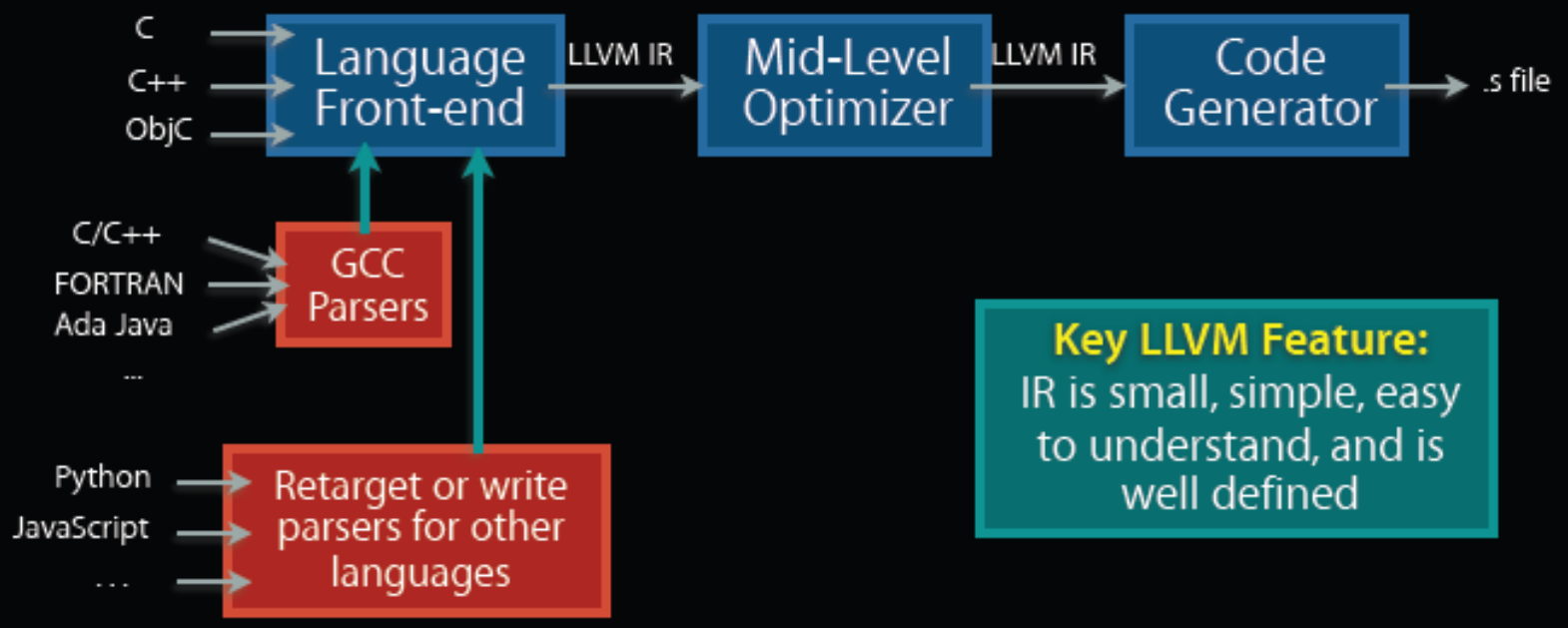
Example of a Simple Static Compiler

- Standard compiler organization, which uses LLVM as midlevel IR:
  - Language specific front-end lowers code to LLVM IR
  - Language/target independent optimizers improve code
  - Code generator converts LLVM code to target (e.g. IA64) code

Many compilers (e.g. GCC) follow this model.

Front-end options for this compiler

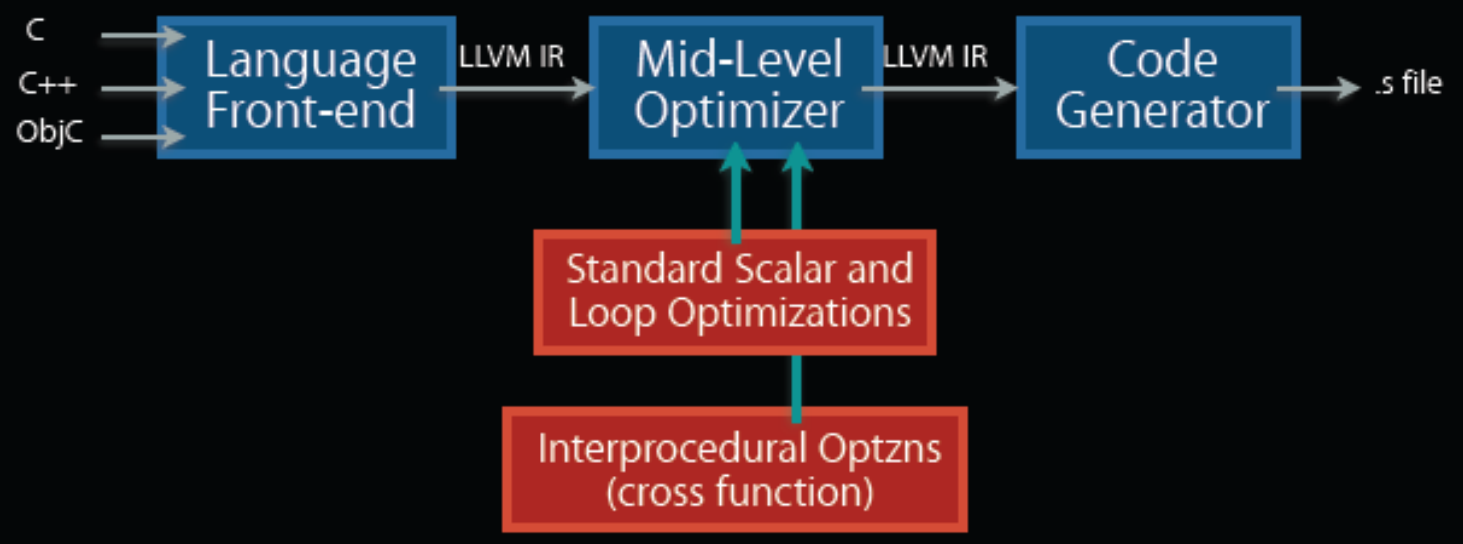
- Front-ends are truly separate from optimizer & codegen
  - Can use front-end AST's that are tailored to the source language
  - Optimizer & Codegen improvements benefit all front-ends
  - Front-ends generate debug info and include it in the IR



llvm-gcc currently uses the GCC 4.0.1 parsers

Optimizer options for this compiler

- Optimizer is solely concerned with semantics of LLVM IR
  - Optimizer & Codegen only know LLVM, not all source languages
  - LLVM includes IP framework and aggressive IP optimizations
  - LLVM uses a modern and light-weight (fast) SSA-based optimizer



编译技术的应用

- 高级程序设计语言的实现
  - 语言特征与编译优化
  - 动态编译
- 针对计算机体系结构的优化
  - 并行性
  - 内存层次结构
- 新计算机体系结构的设计
  - RISC、专用体系结构
- 程序翻译
  - 二进制翻译、硬件合成、数据查询解释器、编译-模拟
- 软件生产力工具
  - 类型检查、边界检查、内存管理工具

1.2 图灵机、冯·诺依曼结构

- 作为计算的抽象数学模型与系统结构模型，图灵机模型、冯·诺依曼机回答了一些与编译相关的根本问题，比如编译的目标是什么，编译为什么是不可替代的，等等。
- 图灵机是由英国数学家阿兰·图灵（A. M. Turing）于1936年提出的一种抽象计算模型，即将人们使用纸笔进行数学运算的过程进行抽象，由一个虚拟的机器替代人们进行数学运算。

图灵机的代数定义

一个确定图灵机（deterministic Turing Machine, DTM）可抽象为一个七元组 $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$，各部分含义如下：

(1)有限状态集合 Q：图灵机的有限个状态，包含初始状态和终止状态。

(2)输入字母表 Σ：允许输入的符号集合，不含空白符 B。

(3)带字母表 Γ：纸带上允许出现的所有符号集合，有 $\Sigma \subseteq \Gamma \wedge B \in \Gamma$。

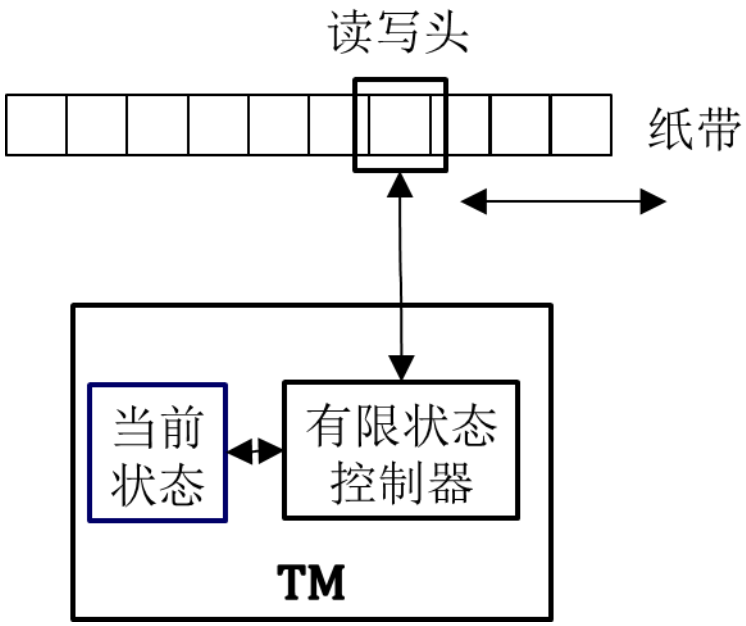
(4)转移函数 $\delta : Q \times \Gamma \mapsto Q \times \Gamma \times \{L, R\}$。$\langle q, s, q', s', d\rangle \in \delta$表示若机器处于状态 $q \in Q$，读写头读到符号 $s \in \Gamma$，则转移到状态 q'，将当前格子的符号改写为 s'，读写头向 $d \in \{L, R\}$ 移动一格（L 向左 R 向右）。

(5)初始状态 q_0：图灵机的唯一初始化当前状态，$q_0 \in Q$。

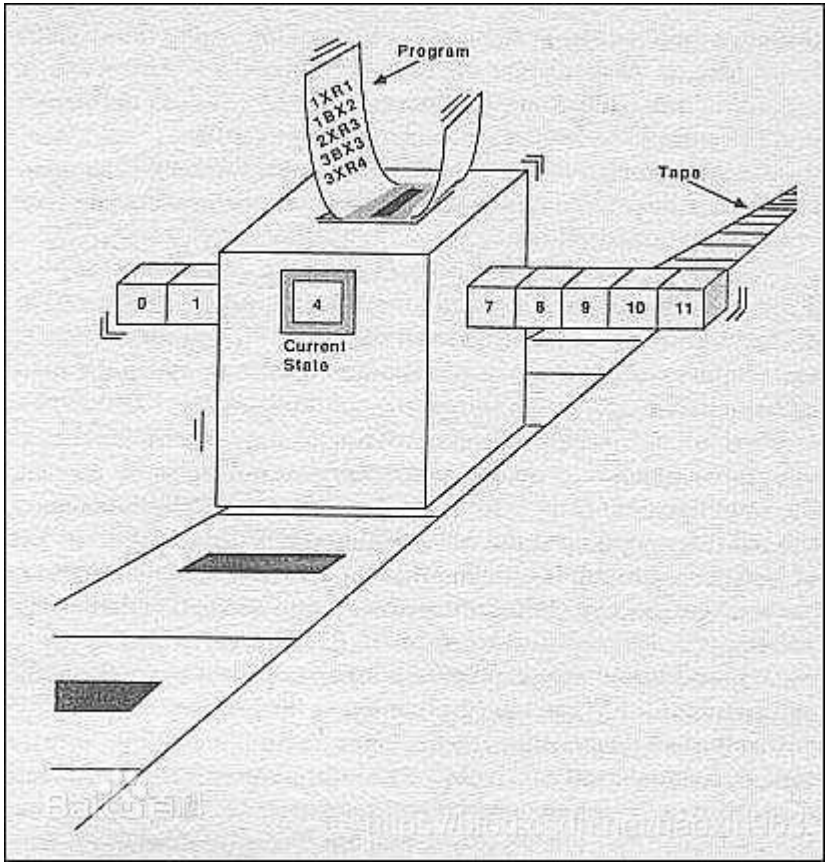
(6)空白符 B：纸带未输入内容区域的默认符号。

(7)终止状态集合 F：$F \subseteq Q$，一旦进入该集合，计算停止。

从图灵机认识抽象计算模型



初始状态
终止状态
当前状态
状态转换规则集
有限状态集
有限字母表



无限长纸带
以读写头方式访问格子

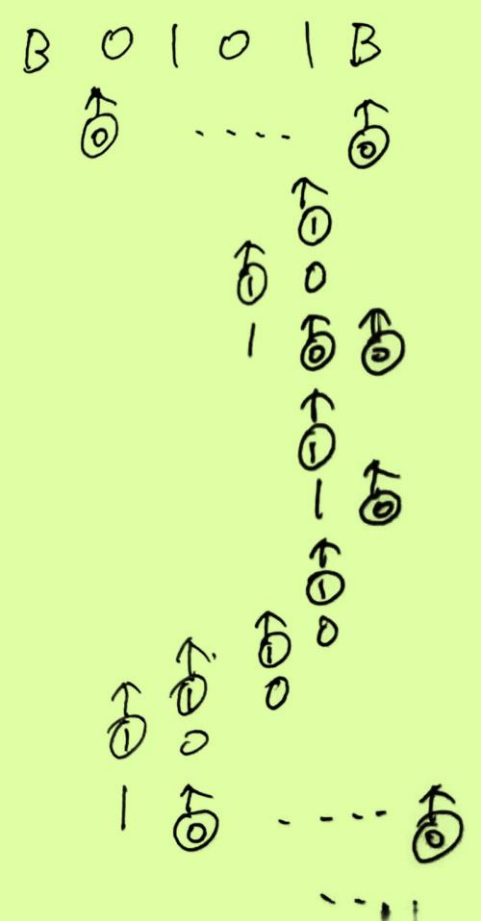
图灵机运行过程

- 图灵机从初始状态 q0开始，按照转移函数δ的规则逐步执行状态转移、符号改写和读写头移动。
- 当读写头进入终止状态集合F时，计算终止，此时纸带上的符号即为计算结果。
- 可计算性判定：一个问题是 “可计算的”，当且仅当存在一个图灵机可以在有限步骤内解决该问题。
- 通用性：存在通用图灵机，它可以模拟任意其他图灵机的计算过程，这是现代通用计算机的理论原型。

例：计数器图灵机

```
(0, 1) -> (0, 1) R
(0, 0) -> (0, 0) R
(0, B) -> (1, B) L

(1, 0) -> (0, 1) R
(1, 1) -> (1, 0) L
(1, B) -> (0, 1) R
```



无符号数加1图灵机：第4、第6条规则中的R改为H，其它不变。

通用图灵机

通用图灵机是图灵机，它能模拟任意图灵机。

具体地说，如果把任意一个图灵机的指令集用图灵于1936–1937年自己提出的一种规范方式编码并预存在纸带上，那么通用图灵机就能够根据纸带上已有的信息，在纸带的空白处模拟那台图灵机的运作，输出那台图灵机应该输出的结果。

通用图灵机被认为是存储程序计算机（冯诺依曼结构1946年）的起源。

高效通用图灵机定理：存在通用图灵机 U，使得对于任意的 $\alpha, x \in \{0,1\}^*$，均有 $U(\alpha,x)=M_\alpha(x)$。而且，如果 $M_\alpha(x)$ 的时间函数是 T，则 $U(\alpha,x)$ 的时间函数是 $O(|\alpha|^c T\log T)$，其中 c 是一个与 α, x 无关的常数。

通用性：存在一个固定的图灵机 U，对于任意图灵机 M 的编码 α 和任意输入 x，U 能精确模拟 M 在 x 上的所有计算步骤（包括状态转移、读写操作）。

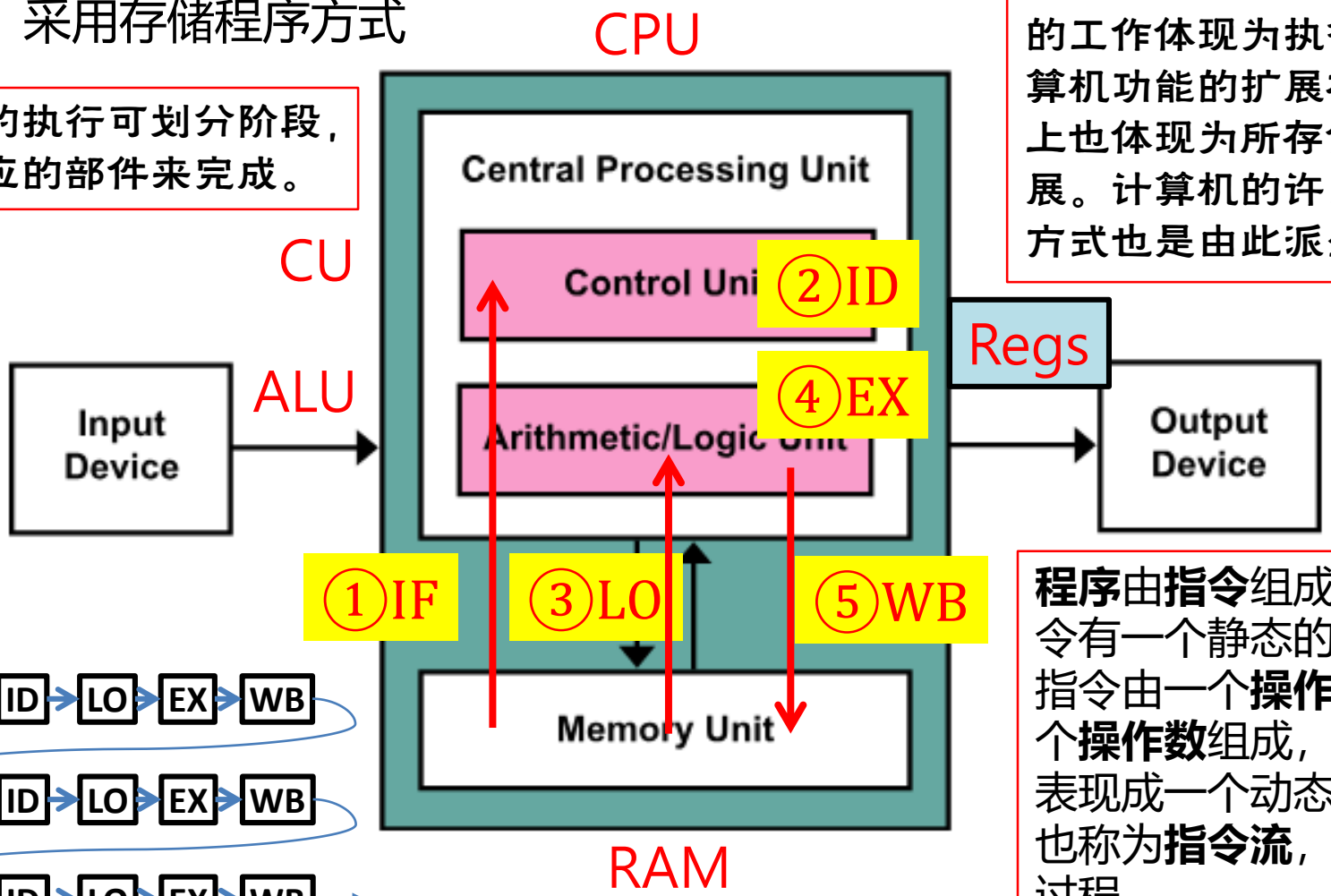
- 普通图灵机DTM M 是针对特定问题设计的（如计算加法、判定字符串是否为回文），而通用图灵机UTM U 是一台“可编程”的图灵机，被认为是存储程序计算机的起源。
- 根据丘奇–图灵论题，UTM的计算能力与DTM、NTM完全等价。最重要的，通用图灵机也是普通图灵机，这就是计算的抽象性的根本所在。
- 冯·诺依曼结构是通用图灵机的架构实现，是目前主流编译程序的目标机的结构模型。

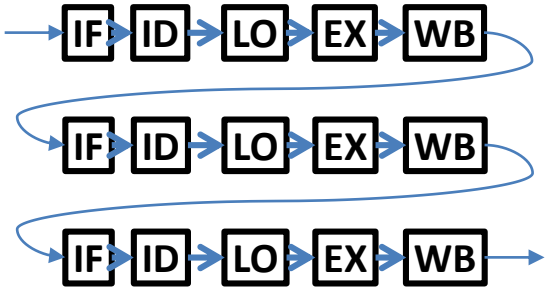
冯·诺依曼结构

- 采用二进制形式表示数据和指令
- 采用存储程序方式

意味着事先编制程序，事先将程序(包含指令和数据)存入内存，运行时自动地、连续地从存储器中依次取出指令且执行。这是计算机能高速自动运行的基础。计算机的工作体现为执行程序，计算机功能的扩展在很大程度上也体现为所存储程序的扩展。计算机的许多具体工作方式也是由此派生的。

指令的执行可划分阶段，由对应的部件来完成。





程序由**指令**组成，其中各指令有一个静态的顺序。一条指令由一个**操作符**和0到多个**操作数**组成，程序的执行表现成一个动态的指令序列，也称为**指令流**，等于PC变化过程。

- 目前除了非冯·诺依曼结构（如哈佛结构、数据流架构），基于冯·诺依曼结构，发展出的多种硬件机器，如ARM、X86、RISC-V、MIPS等各种指令架构，以及针对于冯·诺依曼瓶颈的优化成果。
- 考虑到编译器要与这些硬件机器的细节打交道，所以编译器的目标机往往是多种多样的

1.3 形式语言概论

- 形式语言（Formal Language）理论是计算机科学的基础理论之一，它源于对“计算”与“语言”本质的数学探索，带有严密的数学和逻辑学特征。
- 研究对象是字母表上符合严格文法规则的有限长符号串集合，在词法、句法的精确性，语义可独立定义等方面形成体系。
- 该理论体系的形成，标志着人类从机械的符号处理迈向了形式化、自动化的语法分析与生成。

1.3.1 本课程中心概念

1. 符号
2. 字母表
3. 符号串
4. 语言
5. 问题

1. 符号

- 论域是实体的集合，其中有指代物的实体是符号。

  a letter, figure, or other character or mark or a combination of letters or the like used to designate something.

- 符号有两个属性：名字（用来区分符号本身）、指代。
- 相同名字的符号有可能出现在多个论域中。
- 论域是静态的，负责回答“哪些东西是实体”。
- 上下文是动态的，负责回答“符号当前指代哪个实体”。

术语区分

- 指称 / 指代：符号指向实体的关系本身
- 绑定：符号与实体建立关联这件事
- 赋值：改变绑定的动作

2. 字母表

- 字母表是感兴趣的符号的非空有限集。
- 一般是论域中的所有符号的集合（必须是有限集）

- 例：常见字母表：
  - 二进制数字集合，$\Sigma_{\mathrm{B}} = \{0, 1\}$。
  - 所有小写拉丁字母集合，$\Sigma_a = \{a, b, \ldots, z\}$。
  - 所有ASCII字符的集合。
  - 空白字符的集合，$S = \{\backslash\mathrm{t}, \backslash\mathrm{n}, \backslash\mathrm{r}, \backslash 0, \backslash\mathrm{s}\}$。
  - 字母加特殊符号$构成的集合，$\Sigma_{a\$} = \Sigma_a \cup \{\$\}$。
  - 文法符号的集合（文法中所有终结符和非终结符的集合）

3. 符号串

- 符号串是构建在字母表上的最为基础的结构。
- 一个符号串是由字母表中符号组成的有限长度的序列。
  - $abfbz$ 是 Σ_a 上的一个符号串（简称串）。
  - 90221 是 Σ_{D} 上的一个串， $\Sigma_{\mathrm{D}} = \{0, 1, ..., 9\}$。
  - $ac\$bc$ 是 $\Sigma_{a\$}$ 上的一个串。
  - $\boldsymbol{i}+\boldsymbol{i}\times\boldsymbol{i}$ 是主文法的**文法符号集**上的符号串。
  - $\mathrm{if}(B)S\mathrm{else}S$ 也如上。

- Σ 上的所有符号串组成一个集合记为 Σ^*（克林闭包）

符号串（续）

- 符号串的元素、元素的前驱、元素的后继、元素的索引。
- 一个符号串中的不同元素也可以是同一个符号，对应的元素会被称为这个符号的一次出现。
- 符号串长度：若 w 为一个符号串，$|w|$ 为 w 的长度，
- 长度为0的符号串专用 ε 表示。

- 函数 $\pi(w, i)$ 为 w 的 i前缀，$\pi(w)$ 为所有前缀组成的集合
- 函数 $\tau(w, i)$ 为 w 的 i后缀，$\tau(w)$ 为所有后缀组成的集合

4. 语言

- Σ 上的语言是 Σ 上符号串的一个集合
- 显然，Σ 上的语言是 Σ^* 的一个子集
- 特点，Σ 必须是有限集，而 Σ 上的语言可以是有限集也可以是无穷集。

L_1 是Σ_a上的含有子串“to”的所有串

stop, to, toe are in L_1

e, oyster are not in L_1

$L_1 = \{x \in \Sigma_a^* \mid x$ 含有子串“to”$\}$

语言的例子

$L_2 \quad = \{x \in \Sigma_{\mathrm{D}}{}^* \mid x \text{ 被7整除}\}$

$\quad\quad = \{7, 14, 21, \ldots\}$

$L_3 \quad = \{s\$s \mid s \in \Sigma_a{}^*\}$

$ab\$ab$ in L_3

$ab\$ba$ not in L_3

$a\$\$a\$$ not in L_3因为$a\$$不属于$\Sigma_a{}^*$

5. 问题

- 要求解的问题都是判定性的问题

给定词s, 该词包含“to”子串吗?
给定整数n, 它能被7整除吗?
给定符号串s 和 t, 它们相同吗?

假定字母表是Σ_1，问题即 $s \in L_1$?

假定字母表是Σ_2，问题即 $n \in L_2$?

问题即$|s|==|t|$且 s、$\$$与t依次连接起来是否属于L_3？

问题即 $s\$t \in \{x\$x \mid x \in \Sigma^*, \$ \notin \Sigma\} = L_3$？

判定性问题的答案为是或否。

任何其它问题如查找、统计等功能性问题都可以等价转化为判定性问题（只考虑花费多项式时间）。

1.3.2 形式语言概念

- **形式语言**通常以重写规则 $\alpha \longrightarrow \beta$ 组成的形式文法进行表达，即当符号串 α 出现时可将其替换为符号串 β。
- 从 一个初始的符号串开始，按照不同的顺序，不断应用不同的重写规则，可以得到不同的新符号串。
- 那些不可重写的新符号串全集作为该语言。
- 说这些重写规则定义了这个形式语言。

形式文法

- 一个开始符号作为初始符号串。以终结符和非终结符集合作为字母表。开始符号是非终结符。仅由终结符构成的串是不可重写的串。
- 产生式规则指定了某些符号组合如何被另外一些符号组合替换。

$$S \longrightarrow aSb$$

$$S \longrightarrow ba$$

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aababb$$

$$ba, abab, aababb, aaababbb, \ldots$$

形式文法

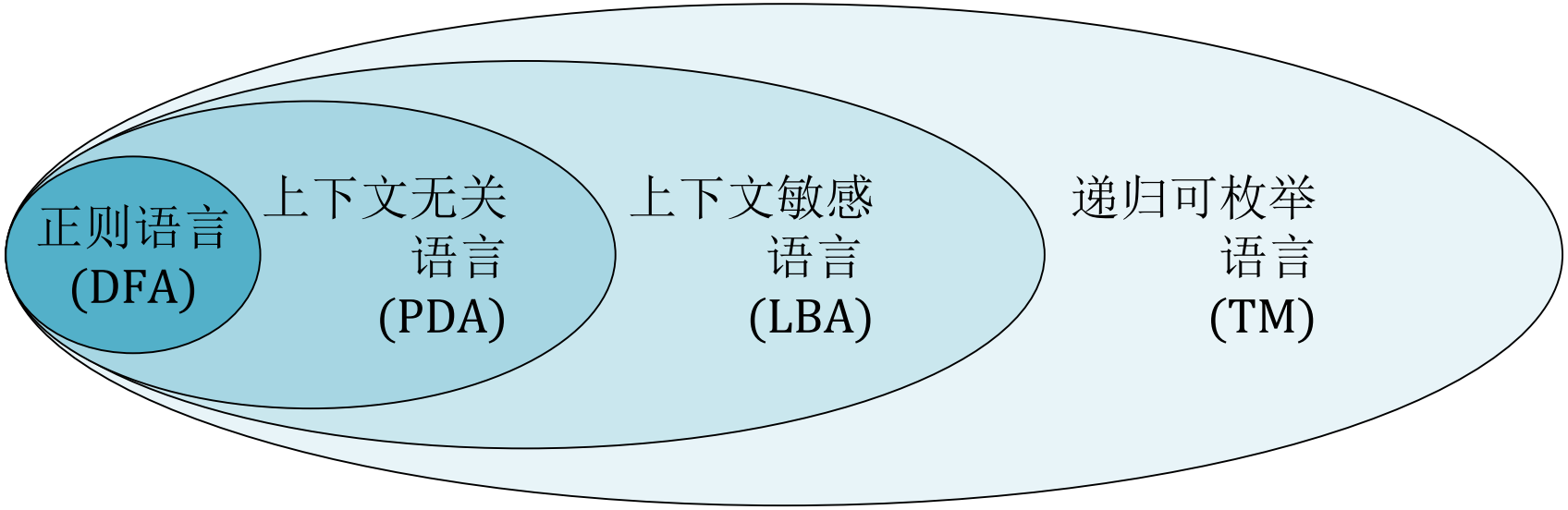
- 一个开始符号作为初始符号串。以终结符和非终结符集合作为字母表。开始符号是非终结符。仅由终结符构成的串是不可重写的串。
- 产生式规则指定了某些符号组合如何被另外一些符号组合替换。

$$S \longrightarrow \text{if}(B)S\text{else}S$$
$$B \longrightarrow d>i$$
$$S \longrightarrow s$$

$$S \Rightarrow \text{if}(B)S \text{ else } S$$
$$\Rightarrow \text{if}(d>i)S \text{ else } S$$
$$\Rightarrow \text{if}(d>i)s \text{ else } S$$
$$\Rightarrow \text{if}(d>i)s \text{ else } s$$

Chomsky 体系

计算机科学中刻画形式文法表达能力的一个分类谱系，是由诺姆·乔姆斯基于1956年提出的。



III型文法	II 型文法	I 型文法	0型文法
$A \longrightarrow \varepsilon \mid a \mid cB$	$A \longrightarrow \gamma$	$\alpha A \beta \longrightarrow \alpha \gamma \beta$	无限制文法

形式语言与形式文法

- 在计算机科学中，形式语言是字母表上，一些有限长符号串的集合。
- 形式文法是描述这种语言的一种途径。

- Languages: “语言是句子的集合，句子是有穷长度的且由某个有穷字母表中的符号构成”
- Grammars: “文法可以被看做是一个装置，它恰好枚举一个语言的句子

- *N. Chomsky, Information and Control, Vol 2, 1959*

An alphabet is a set of symbols:

{0,1}

Sentences are strings of symbols:

0,1,00,01,10,1,...

A language is a set of sentences:

L = {000,0100,0010,..}

A grammar is a finite list of rules defining a language.

$S \longrightarrow 0A$	$B \longrightarrow 1B$
$A \longrightarrow 1A$	$B \longrightarrow 0F$
$A \longrightarrow 0B$	$F \longrightarrow \varepsilon$

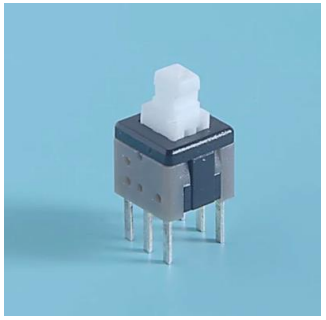
历史回顾

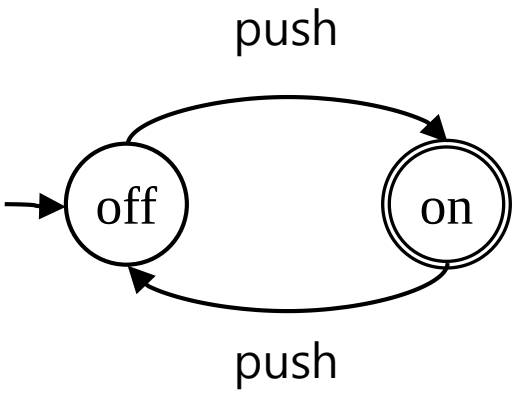
- 奠基期（1930s-1940s）：逻辑与自动机的萌芽
- 阿兰·图灵于1936年提出的图灵机模型，为一切可计算过程提供了形式化定义。这不仅是计算机的理论原型，也隐含了“语言识别”的思想。
- 同时期，波斯特产生式系统 和 丘奇-图灵论题 共同奠定了计算理论的基础，其中“重写规则”已成为核心操作。

- 成型与飞跃期（1950s-1960s）：乔姆斯基谱系的建立。
- 1956年，语言学家诺姆·乔姆斯基（Avram Noam Chomsky）为研究自然语言语法，系统性地提出了形式文法的概念，并依据产生式规则的约束强弱，划定了著名的乔姆斯基谱系（0-3型文法）。这一分类奇迹般地与不同计算能力的自动机模型（图灵机、线性有界自动机、下推自动机、有限自动机）完美对应。

1.3.3 有穷自动机理论

- 在理论计算机科学中，自动机理论是对抽象机器和它们能解决的问题的研究。
- 自动机是有限状态机器，通过状态之间的转移来消耗掉输入串，以判定是否为自动机所接受。
- 设想有穷自动机运行过程中从左到右逐一符号地读输入串，根据输入符号进行状态转移，一旦输入串被读完，自动机就“停止”了。
- 根据自动机停止时的状态是不是“接受”状态，就得到自动机是否接受这个符号串的结论。
- 自动机接受的符号串的全集被称为“这个自动机接受的语言”。

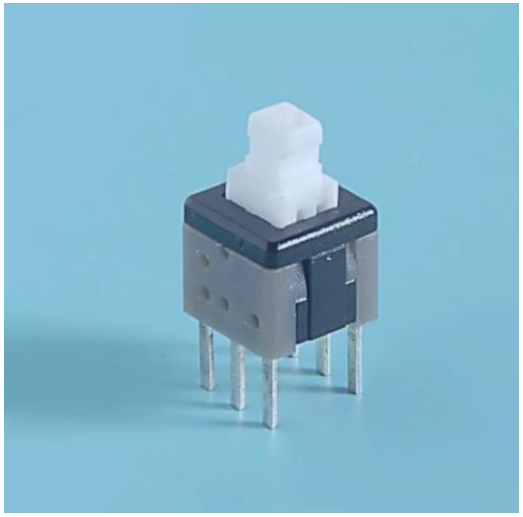




可见，有穷自动机、形式文法都能用于定义形式语言

两相开关的DFA建模

建模对象：两相开关，是一个物理系统。

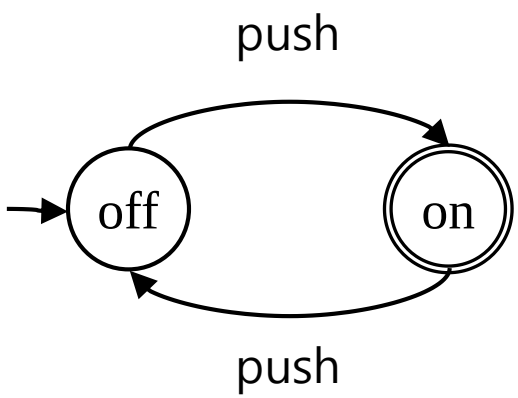


- 有“开”和“关”两个状态；
- 外界按压（push）表示在系统边界上发生的交互行为；
- 开始时开关的状态为“开”；
- 一个push导致状态“开”、“关”切换一次；
- 一到多个push之后开关的状态是“开”，则达到接通之目的。

建模结果：一个DFA

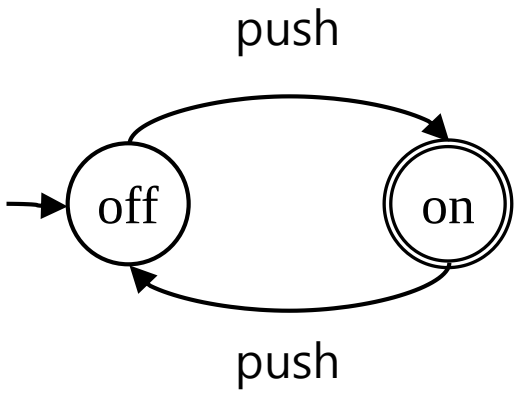
两相开关DFA

- 建模结果



- 两个状态on、off
- 初始状态off
- 接受状态on
- 状态转移弧
  (off, push) -> on
  (on, push) -> off
- 转移弧上的标记push
- 输入符号push

两相开关DFA运行过程



- ✓ 给定输入串，由单个符号push组成的符号串；
- ✓ 从初始状态开始，并将输入串第一个符号作为当前输入符号；
- ✓ 对于当前输入（push表示）发生状态转移；
- ✓ 当输入串消耗完自动机到达on则接受；
- ✓ 否则不接受；
- ✓ 本模型中接受表示物理系统接通，否则是断开的。

一些类型的“抽象机器”

有穷自动机	有少量存储的装置 用于建模非常简单的事情	识别复杂度：线性
下推自动机	有无限量存储并以受限方式访问的装置 可用于语法分析	识别复杂度：多项式
图灵机	有无限量存储的装置 这是实际的计算机	
时间界限自动机	存储器无限大，但运行时间有界 这是以合理速度运行的计算机	a type of Turing machine with a tape of finite length, bounded by the length of the input string

1.3.4 形式语言定位

- 人工语言：人类有目的地构建一些communication systems和语言，用于非常专门的任务。（不同于自然语言）
  - 公元前三百年Aristotle的命题逻辑
  - 十一世纪**圭多·达·阿列佐发明音乐记谱法**（四线谱）
  - 各种人机交互语言，如计算、搜索、器人控制机等
  - 物-物接口：如PDF
- 任意被设计成的语言都是人工的，有其定义。但只有其中一些是形式语言。
  - 比如Java是形式化的，而世界语是由人设计的但不是形式化的。

形式语言的等价性

自动机	形式文法	形式语言
DFA	正则文法	正则语言
PDA	CFG	CFL
TM	0型文法	递归可枚举语言

- 自动机便于执行
- 形式文法便于推导、推理、生成
- 形式语言便于枚举
- 都称为语言识别器
- 本课程讨论范围为表的前两行所示

形式语言理论的应用

- 在软件工程与安全领域：程序分析与验证：用于建模程序行为（如基于文法的模糊测试）、规范协议与API（如使用形式文法定义通信协议或文件格式），以及检测安全漏洞（如SQL注入、XSS攻击的模式匹配）。
- 模型驱动开发：领域特定语言（DSL）的设计与实现，其核心即是利用形式文法理论定义DSL的语法。
- 在人工智能与自然语言处理领域：结构预测与深度学习的结合：传统的文法模型能为神经网络提供可解释的结构化约束，结合深度学习强大的表示学习能力，共同提升句法分析、信息抽取等任务的性能与可解释性。
- 神经符号计算：探索如何将形式化符号系统（文法、逻辑）与神经网络相融合，构建兼具逻辑严谨性与学习泛化能力的新一代AI模型。
- 在生物信息学与新兴交叉学科：DNA/蛋白质序列分析：形式语言被用于建模生物序列的模式和结构，因为生物分子序列本质上也是一种符号串。
- 量子计算语言：为新兴的量子编程语言设计语法和语义提供形式化基础。

1.4 补充：关于形式化证明

- 形式化证明很有用，在后续课程中用起来
  - 演绎证明
  - 归纳证明：数学归纳法
  - 反证法
- 鸽巢原理
- 直接运用先修课的知识

数学归纳法

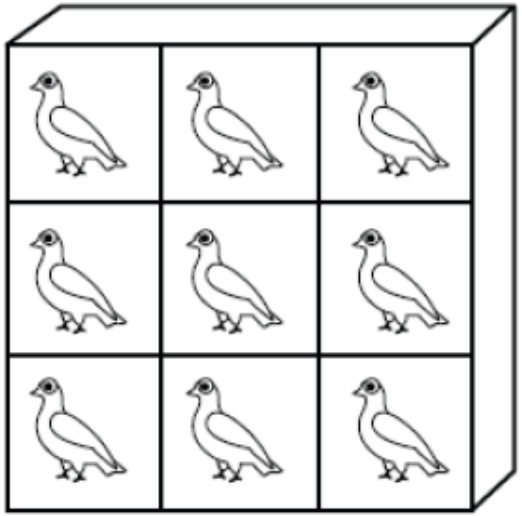
Example

Prove that $1^2 + 2^2 + 3^2 + \ldots + n^2$ is $\frac{n(n+1)(2n+1)}{6}$.

- **Basis (n=1):** $\frac{1(1+1)(2\cdot 1+1)}{6} = \frac{2\cdot 3}{6} = 1 = 1^2$.
- **Induction:** Let $\mathcal{P}(n) := 1^2 + \ldots + n^2$ is $\frac{n(n+1)(2n+1)}{6}$.
  $\mathcal{P}(n+1) = 1^2 + \ldots + n^2 + (n+1)^2 = \frac{n(n+1)(2n+1)}{6} + (n+1)^2$
  $= (n+1)\left\{\frac{n(2n+1)+6n+6}{6}\right\} = (n+1)\left\{\frac{2n^2+7n+6}{6}\right\}$
  $= (n+1)\left\{\frac{2n^2+4n+3n+6}{6}\right\} = \frac{(n+1)(n+2)(2n+3)}{6}$
- Since $\mathcal{P}(1)$ is true and $\mathcal{P}(n) \Rightarrow \mathcal{P}(n+1)$, we conclude that $\mathcal{P}(n)$ is true for all $n \geq 1$.

鸽巢原理

如果有n+1个鸽子和n个鸽巢，那么某个鸽巢必须至少住进两只鸽子。



如果有kn+1个鸽子和n个鸽巢，那么某个鸽巢必须至少住进k+1只鸽子，k>=1.

Pigeonhole Principle

If there are $n+1$ pigeons and n pigeonholes, then some pigeonhole must contain at least two pigeons.

其它预备知识

- 图论
  - 有向图、无向图
  - 结点的度、入度、出度
  - 路径、路径长度、环
- 集合论
  - 集合运算及性质
  - 关系、映射、函数
  - 划分、覆盖
- 数据结构
  - 线性表、树、杂凑表、链表
  - 操作、算法及其复杂度

小结与作业

- 知识点：编译相关术语，编译过程，图灵机模型，有穷自动机构成及原理，形式文法概念，符号、字母表、符号串、语言、问题等概念。

- 1.7习题：1.3、1.5(2)、1.7(2)
- 阅读：教材第一章